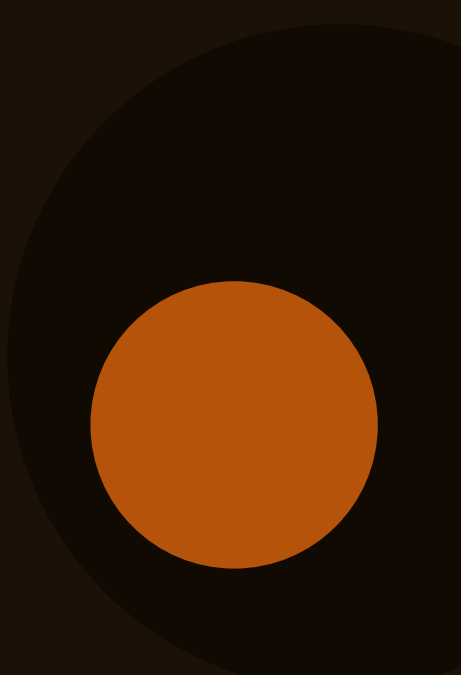




JAVA

Primeiros Passos em Java

Do JShell à fluência com os tipos primitivos da linguagem

Apostila completa do zero, com exemplos práticos e explicações passo a passo dos fundamentos da linguagem.

CONTEÚDO DA APOSTILA

- 01 JShell e o Primeiro Programa
- 02 Variáveis e Palavras-chave
- 03 Tipos Inteiros
- 04 Tipos de Ponto Flutuante
- 05 char, boolean e String
- 06 Operadores e Expressões
- 07 Casting e Conversão de Tipos

JavaStudiesHub

Sumário

0
1

JShell e o Primeiro Programa

O ambiente interativo do Java e a anatomia de um statement

0
2

Variáveis e Palavras-chave

Como o Java guarda dados na memória e como nomear as coisas

0
3

Tipos Inteiros

byte, short, int e long — tamanho, alcance e overflow

0
4

Tipos de Ponto Flutuante

float e double — precisão e notação científica

0
5

char, boolean e String

Caracteres, valores lógicos e o texto imutável do Java

0
6

Operadores e Expressões

Aritmética, módulo, incremento e operadores compostos

0
7

Casting e Conversão de Tipos

Widening automático e narrowing com cast explícito

01

JShell e o Primeiro Programa

Explorando a linguagem Java linha a linha, sem projeto e sem compilação manual

Para que serve o JShell

Antes de existir o JShell, aprender Java significava criar uma classe inteira só para testar uma linha de código. Desde o JDK 9, o JShell resolve esse atrito: é um console interativo (um REPL — read, eval, print, loop) onde cada instrução digitada é executada imediatamente, com o resultado aparecendo na hora. Não é preciso declarar uma classe, escrever um método `main` nem compilar arquivo nenhum — é o espaço ideal para testar uma ideia rápida ou entender como um trecho de código se comporta.

No dia a dia do JShell, as setas `↑` e `↓` navegam pelo histórico de comandos já digitados, o comando `/vars` lista todas as variáveis criadas na sessão atual, e `/exit` (ou o atalho `/ex`) encerra a sessão.

O clássico Hello World

Todo aprendizado de uma linguagem nova costuma começar imprimindo uma mensagem na tela. Em Java, isso é feito com o método `println`, que pertence à classe `System` e imprime o texto seguido de uma quebra de linha. Já o método irmão `print` faz a mesma coisa, mas sem pular linha ao final — útil quando você quer concatenar várias saídas na mesma linha.

```
System.out.println("Bem-vindo à TechStore!");  
// Saída: Bem-vindo à TechStore!  
  
// println pula linha; print não pula:  
System.out.print("Estoque: ");  
System.out.print("42 unidades");  
// Saída: Estoque: 42 unidades (tudo na mesma linha)
```

O que caracteriza um statement

Um statement é uma instrução completa — uma unidade de trabalho que o Java executa do início ao fim. Assim como uma frase termina com ponto final, todo statement em Java termina com ponto e vírgula. É esse sinal que avisa ao compilador onde uma instrução acaba e a próxima começa; esquecer-lo é, de longe, um dos deslizes mais comuns de quem está começando.

Vale reforçar também que Java diferencia maiúsculas de minúsculas (é case-sensitive). Isso significa que `System`, com S maiúsculo, é a classe correta — já `system`, com s

minúsculo, simplesmente não existe para o compilador e gera erro.

DICA

O JShell é o lugar perfeito para arriscar. Como não existe um projeto inteiro em jogo, um erro custa apenas uma linha — corrija na sequência e siga em frente. Errar ali faz parte do processo de internalizar a sintaxe.

02

Variáveis e Palavras-chave

Declarando espaço na memória e respeitando o vocabulário reservado da linguagem

O que realmente é uma variável

Pense em uma variável como uma gaveta rotulada dentro da memória do computador: você escolhe o rótulo (o nome), define que tipo de coisa cabe ali (o tipo de dado) e, opcionalmente, já guarda um valor dentro dela. O Java cuida da parte de reservar o espaço físico na RAM — você só precisa declarar a intenção.

Toda variável em Java carrega três elementos: o tipo (o que pode ser armazenado ali), o nome (como o código vai se referir a ela) e, de forma opcional, um valor inicial atribuído no momento da criação.

```
// tipo nome valor
int quantidadeEstoque = 40; // declaração completa
double precoUnitario = 199.90;
boolean emPromocao = true;
String nomeProduto = "Fone Bluetooth";

// Alterando o valor depois – sem repetir o tipo:
quantidadeEstoque = 35;
```

Palavras-chave: o vocabulário que já pertence à linguagem

Palavras-chave (keywords) são termos que o próprio Java reserva para usos específicos — `int`, `class`, `if`, entre outras. Justamente por já terem um significado fixo, elas não podem batizar variáveis, classes ou métodos. E, como o Java é sensível a maiúsculas e minúsculas, apenas a grafia exata em minúsculo conta como palavra-chave: `int` é reservada, mas `Int` não é — seria um nome válido para uma classe, por exemplo. A maioria dos editores, incluindo o IntelliJ, colore as keywords de forma diferente do resto do código, o que ajuda a identificá-las rapidamente.

REGRA — Convenção de nomes

A comunidade Java segue o padrão camelCase para variáveis e métodos: a primeira palavra começa em minúsculo e cada palavra seguinte inicia com maiúscula, sem espaços — `precoProduto`, `calcularDesconto`, `isDisponivel`. Seguir essa convenção deixa o código reconhecível para qualquer outro desenvolvedor Java.

03

Tipos Inteiros

byte, short, int e long — escolhendo o tamanho certo para números sem casas decimais

Quatro tamanhos para números inteiros

Quando o valor não precisa de casas decimais, o Java oferece quatro tipos primitivos, cada um reservando uma quantidade diferente de bits na memória — e, por consequência, cobrindo uma faixa diferente de valores possíveis. Escolher o tipo certo evita duas armadilhas opostas: desperdiçar memória usando um tipo maior do que o necessário, ou provocar overflow usando um tipo pequeno demais para o valor real.

Tipo	Bits	Faixa de valores	Quando escolher
byte	8	-128 a 127	Dados binários, protocolos de rede
short	16	-32.768 a 32.767	Uso raro no Java moderno
int	32	aprox. $\pm 2,14$ bilhões	Padrão para a maioria dos inteiros
long	64	aprox. $\pm 9,2 \times 10^{18}$	IDs muito grandes, timestamps

Overflow e underflow: quando o número dá a volta

Overflow acontece quando um cálculo ultrapassa o valor máximo que o tipo consegue representar. Diferente de outras linguagens, o Java não lança um erro nessa hora — o valor simplesmente "gira" e volta para o limite mínimo do tipo, silenciosamente. O underflow é o mesmo problema na direção oposta, abaixo do valor mínimo. É um comportamento traiçoeiro justamente porque o programa continua rodando, só que com um número que não faz mais sentido. As constantes `Integer.MAX_VALUE` e `Integer.MIN_VALUE` ajudam a checar esses limites antes que eles causem um bug silencioso.

```
int estoqueMaximo = Integer.MAX_VALUE;
System.out.println(estoqueMaximo); // 2147483647
System.out.println(estoqueMaximo + 1); // -2147483648 (deu a volta!)

// O sufixo L é obrigatório em literais long acima da faixa do int:
long faturamentoAnual = 8_100_000_000L;
```

```
// Underscores (Java 7+) só existem para facilitar a leitura:  
int totalClientesCadastrados = 215_000;
```

04

Tipos de Ponto Flutuante

float e double — como o Java representa números com casas decimais

Dois tipos, duas precisões

Para números com parte decimal, o Java disponibiliza dois primitivos: **float**, com 32 bits e precisão simples, e **double**, com 64 bits e precisão dupla. Na prática, double é o tipo padrão da linguagem para qualquer decimal — sempre que um número como 3.14 é escrito sem indicação extra, o compilador já assume que ele é double.

Usar float exige um detalhe fácil de esquecer: o sufixo f (ou F) no final do literal. Sem ele, o compilador interpreta o número como double e recusa a atribuição por incompatibilidade de tipo. Na prática, double costuma ser a escolha mais segura, até porque os métodos da classe Math (como sqrt e pow) trabalham nativamente com double.

```
double margemLucro = 0.32159265358979; // double é o padrão
float f = 3.14f; // float exige o sufixo f!
// float x = 3.14; // ERRO: 3.14 é double, não float

// Notação científica:
double espessuraChip = 1.4E-10;
double faturamentoGlobal = 3.4E12;

// double é visivelmente mais preciso que float:
float fDiv = 1.0f / 3.0f; // 0.33333334
double dDiv = 1.0 / 3.0; // 0.3333333333333333
```

ATENÇÃO — Dinheiro nunca em float ou double

Números de ponto flutuante representam decimais de forma aproximada — e para cálculos financeiros essa aproximação vira erro de centavo somado ao longo de muitas operações. Para valores monetários, o tipo correto é BigDecimal, que preserva precisão exata.

05

char, boolean e String

O caractere Unicode, o valor lógico e o texto imutável da linguagem

char: um único caractere, três formas de escrever

O tipo **char** guarda exatamente um caractere, sempre entre aspas simples, ocupando 16 bits. Internamente, porém, ele nada mais é do que um número baseado na tabela Unicode — o que abre três formas equivalentes de atribuição: o literal do caractere em si, o código decimal Unicode correspondente, ou o código em hexadecimal.

```
char categoria = 'A'; // literal – a forma mais comum
char codigoBarra = 66; // decimal: 66 corresponde a 'B'
char hex = '\u0043'; // hexadecimal Unicode ('C')

// Cuidado: char + char produz um int, não concatena texto:
char c1 = 'A', c2 = 'B';
System.out.println(c1 + c2); // 131 (65 + 66) – soma numérica!
```

boolean: apenas verdadeiro ou falso

O tipo **boolean** só admite dois valores possíveis: **true** ou **false**. Por convenção, nomes de variáveis boolean costumam começar com is, has ou can — isso faz o código soar como uma pergunta (isDisponivel, hasDesconto, canComprar), o que melhora bastante a leitura.

String: texto que nunca muda depois de criado

Diferente dos primitivos, **String** é uma classe — mas é tratada de forma tão frequente que parece parte da linguagem básica. Uma String representa uma sequência de caracteres imutável: uma vez criada, seu conteúdo não muda mais. Qualquer operação que pareça alterar uma String, na verdade, cria uma String nova. Isso garante segurança em código concorrente e permite que o Java reutilize Strings idênticas através do chamado String Pool, economizando memória.

```
String nomeLoja = "TechStore";

// Concatenação com +:
String aviso = "Bem-vindo à " + "TechStore!"; // "Bem-vindo à TechStore!"
String info = "Estoque: " + 42; // "Estoque: 42" (int vira texto)

// Imutabilidade – cada operação cria uma nova String:
String slogan = "Compre";
```

```
slogan = slogan + " com os melhores preços!"; // "Compre" original não foi alterada
```

06

Operadores e Expressões

Aritmética, resto da divisão e os atalhos de incremento e decremento

Os cinco operadores aritméticos

O Java trabalha com cinco operadores aritméticos básicos: `+` (soma ou concatenação de texto), `-` (subtração), `*` (multiplicação), `/` (divisão) e `%` (resto da divisão, também chamado de módulo). O operador de módulo é mais versátil do que parece: serve para verificar se um número é par (`n % 2 == 0`), para criar ciclos que se repetem dentro de um array, ou para limitar um valor a um intervalo específico.

Incrementar e decrementar sem repetir código

Somar ou subtrair 1 de uma variável é tão comum que o Java oferece um operador dedicado só para isso. As três formas abaixo chegam exatamente ao mesmo resultado — a diferença está na quantidade de código necessária. O pós-incremento (`++`) é o mais compacto e o mais usado no dia a dia.

```
int cliquesBotaoComprar = 10;

// Três formas de incrementar, mesmo resultado:
cliquesBotaoComprar = cliquesBotaoComprar + 1; // forma explícita
cliquesBotaoComprar += 1; // operador composto
cliquesBotaoComprar++; // pós-incremento (o mais idiomático)

// Operadores compostos existem para toda operação:
int pontosFidelidade = 100;
pontosFidelidade += 5; // pontosFidelidade = pontosFidelidade + 5
pontosFidelidade -= 3; // pontosFidelidade = pontosFidelidade - 3
pontosFidelidade *= 2; // pontosFidelidade = pontosFidelidade * 2
pontosFidelidade /= 4; // pontosFidelidade = pontosFidelidade / 4
pontosFidelidade %= 3; // pontosFidelidade = pontosFidelidade % 3
```

ATENÇÃO — Cast implícito nos operadores compostos

Operadores como `+=` e `-=` aplicam um cast implícito de volta para o tipo da variável à esquerda. Isso significa que `byte b = 10; b -= 5.5;` compila normalmente, mas o resultado decimal é truncado para caber num byte — um comportamento fácil de não perceber.

07

Casting e Conversão de Tipos

Quando o Java converte sozinho e quando ele exige uma conversão explícita

Por que a conversão nem sempre é automática

O Java evita converter tipos sozinho sempre que essa conversão puder causar perda de dados. Quando uma operação mistura tipos diferentes, o resultado é automaticamente ampliado para o tipo maior envolvido — somar um byte com um short, por exemplo, resulta em um int, não em um byte. Se você quiser guardar esse resultado de volta num tipo menor, precisa pedir isso explicitamente através de um cast.

Widening automático x narrowing manual

Existem duas direções de conversão. O widening (alargamento) acontece sozinho quando um valor de tipo menor é atribuído a um tipo maior — não há risco de perda, então o Java não exige nada além da atribuição normal. Já o narrowing (estreitamento) vai no sentido contrário e sempre exige um cast explícito, justamente porque pode haver perda de informação. A ordem de tamanho entre os tipos numéricos é: byte < short < int < long < float < double.

```
// WIDENING – automático, sem cast:
int unidadesVendidas = 100;
double mediaVendas = unidadesVendidas; // int para double: ok, sem perda (100.0)

// NARROWING – exige cast explícito:
double valorTotal = 149.99;
int valorArredondado = (int) valorTotal; // cast obrigatório
System.out.println(valorArredondado); // 149 – corta, não arredonda!

// Cast também é necessário em expressões com byte/short:
byte caixa1 = 10, caixa2 = 20;
byte totalCaixas = (byte)(caixa1 + caixa2); // resultado da soma já é int
```

DICA

O cast de double para int sempre trunca a parte decimal — nunca arredonda. Por isso (int) 9.9 vale 9, não 10. Quando o objetivo for arredondar de verdade, use Math.round() antes de converter para int.

Fim de apostila

Este capítulo consolidou as bases da linguagem: tipos primitivos, variáveis, operadores e conversão de tipos.

Esses fundamentos sustentam tudo o que vem depois — lógica condicional, laços de repetição, métodos e orientação a objetos.

JavaStudiesHub